

Auditoría de TourneyPro

Revisión de seguridad, bugs y problemas de diseño — generado el 22/09/2026

Revisión del backend (Express + MySQL), los schemas de validación (zod) y las partes del frontend (React) que consumen esos endpoints. Los hallazgos están ordenados de más a menos graves. Las referencias de archivo:línea corresponden al estado del proyecto en el momento de la revisión.

CRÍTICO

1. updateGameData está roto — guardar resultados de partidos siempre falla

[backend/querys/querys.js:452-453](#)

backend/querys/querys.js:453 usa `req.user.id`, pero el middleware de autenticación solo guarda `req.user_id` en la request (backend/server.js:30). `req.user` es undefined, así que esto lanza un `TypeError` capturado por el `catch`, y **todo intento de guardar un resultado de partido devuelve 500**. Esta es la función que dispara `src/hooks/useSetGameResult.jsx` cuando se rellena el marcador en el bracket.

```
const [canEditGame] = await connection.query(
  "SELECT created_by FROM tournaments WHERE tournament_id = ? AND created_by = ?",
  [tournamentId, req.user.id] // deberia ser req.user_id
);
```

2. tournamentInscription no comprueba plazas de verdad — se pueden inscribir infinitos equipos

[backend/querys/querys.js:400-403](#)

La query cuenta filas de la tabla `tournaments` (siempre 1, porque se filtra por `tournament_id`), no los equipos inscritos en `tournament_teams`. El check `registered_teams >= total` prácticamente nunca se cumple, así que un torneo de 8 plazas puede acabar con 20 equipos inscritos sin que nada lo impida. **Esto explica por qué existían torneos con total_teams que no era potencia de 2** antes de la corrección manual aplicada sobre la base de datos.

```
SELECT COUNT(*) AS registered_teams, total_teams AS total
FROM tournaments WHERE tournament_id = ?
-- deberia ser COUNT(*) FROM tournament_teams WHERE tournament_id = ?
```

3. XSS almacenado vía team_shield en el bracket

[src/hooks/Bracketry.jsx:146](#) · [entrada en src/hooks/CreateTeamForm.jsx:19](#)

`team_shield` es un campo de texto libre sin validar (backend/schemas.js: `team_shield: z.string().optional()`) que el usuario rellena al crear/editar un equipo. Se concatena como HTML crudo y se pasa a la librería `bracketry`, que lo inserta tal cual en el DOM. Cualquiera puede meter un payload como escudo de su equipo y ejecutar JavaScript en el navegador de cualquiera que vea ese bracket, incluidos los organizadores. Las cookies de sesión son `httpOnly` (mitiga el robo directo), pero el script inyectado puede seguir haciendo fetch con `credentials:'include'` y actuar en nombre de la víctima.

```
nationality: `
```

4. tournamentInscription no comprueba que el usuario sea dueño/miembro del equipo

[backend/querys/querys.js:388-416](#)

Cualquier usuario autenticado puede inscribir **cualquier teamId** (no solo los suyos) en cualquier torneo. No hay comprobación de founder_id ni de pertenencia vía team_players antes de insertar en tournament_teams.

IMPORTANTE

5. /api/getDashboardData es pública y expone toda la base de datos de esas tablas

[backend/server.js:61](#) · [backend/querys/querys.js:12-14](#)

La ruta no tiene authMiddleware, y la query hace SELECT * FROM games, SELECT * FROM teams y SELECT * FROM tournaments sin filtros. Cualquiera sin sesión iniciada puede pedir este endpoint directamente y volcar esas tres tablas completas.

6. /api/register no tiene límite de peticiones (rate limit)

[backend/server.js:57-58](#)

Solo /api/login tiene loginLimiterMiddleware aplicado. El registro de usuarios está abierto a spam y creación masiva de cuentas sin ninguna restricción.

7. Enumeración de emails en el registro

[backend/querys/querys.js:130-135](#)

El endpoint responde explícitamente "Este usuario ya existe" cuando el email ya está registrado, lo que permite a un atacante comprobar sistemáticamente qué correos están dados de alta en la aplicación.

8. El logout no invalida el refresh token en la base de datos

[backend/querys/querys.js:257-276](#)

Solo se borran las cookies del navegador; el refresh_token sigue siendo válido en la tabla refresh_tokens hasta que expira (7 días). Si alguien robó el token antes del logout, puede seguir usándolo con normalidad tras cerrar sesión.

MENOR / CALIDAD

9. joinTeam puede bloquear equipos con max_players nulo

[backend/querys/querys.js:759-766](#)

Si max_players es NULL en la base de datos, la comparación total >= null se evalúa como total >= 0 en JavaScript, que es casi siempre true. Resultado: nadie puede unirse a un equipo que no tiene límite de jugadores definido.

10. Sin rutas protegidas centralizadas en el frontend

[src/App.jsx](#)

App.jsx no envuelve las rutas privadas en un componente guard de autenticación; cada página gestiona su propio 401/redirect por separado. El backend sí protege correctamente los endpoints, así que no es un agujero de seguridad real, pero sí un riesgo de mantenimiento: es fácil olvidar el guard al añadir una página nueva.

11. Sin CORS configurado ni cabeceras de seguridad (helmet)

`backend/server.js`

No es grave si frontend y backend sirven desde el mismo origen en producción, pero no hay helmet ni política de Content-Security-Policy. Dado el hallazgo de XSS (#3), una CSP ayudaría a limitar su impacto real incluso si el XSS no se corrige de inmediato.

Recomendación de prioridad: arreglar primero los hallazgos #1 y #2 (críticos, ambos impiden el funcionamiento correcto de la app), seguidos de #3 (XSS) y #4 (autorización de inscripción).